

AN TOÀN THÔNG TIN (INSE330380)

CHƯƠNG II BẢO MẬT PHẦN MỀM VÀ BẢO MẬT HỆ THỐNG

Ths. ĐỖ MINH HẬU

- ☐ Giới thiệu bài học
- ☐ Lỗi phần mềm: vấn đề bảo mật phần mềm. Nguồn gốc của lỗi hỏng phần mềm
- ☐ Lập trình phòng thủ/ An toàn
- ☐ Bố trí bộ nhớ xử lý
- ☐ Lỗi hỏng phần mềm - Buffer overflow: Stack overflow; Heap overflow
- ☐ Phòng chống các cuộc tấn công Buffer overflow
- ☐ Tóm tắt
- ☐ Quizze

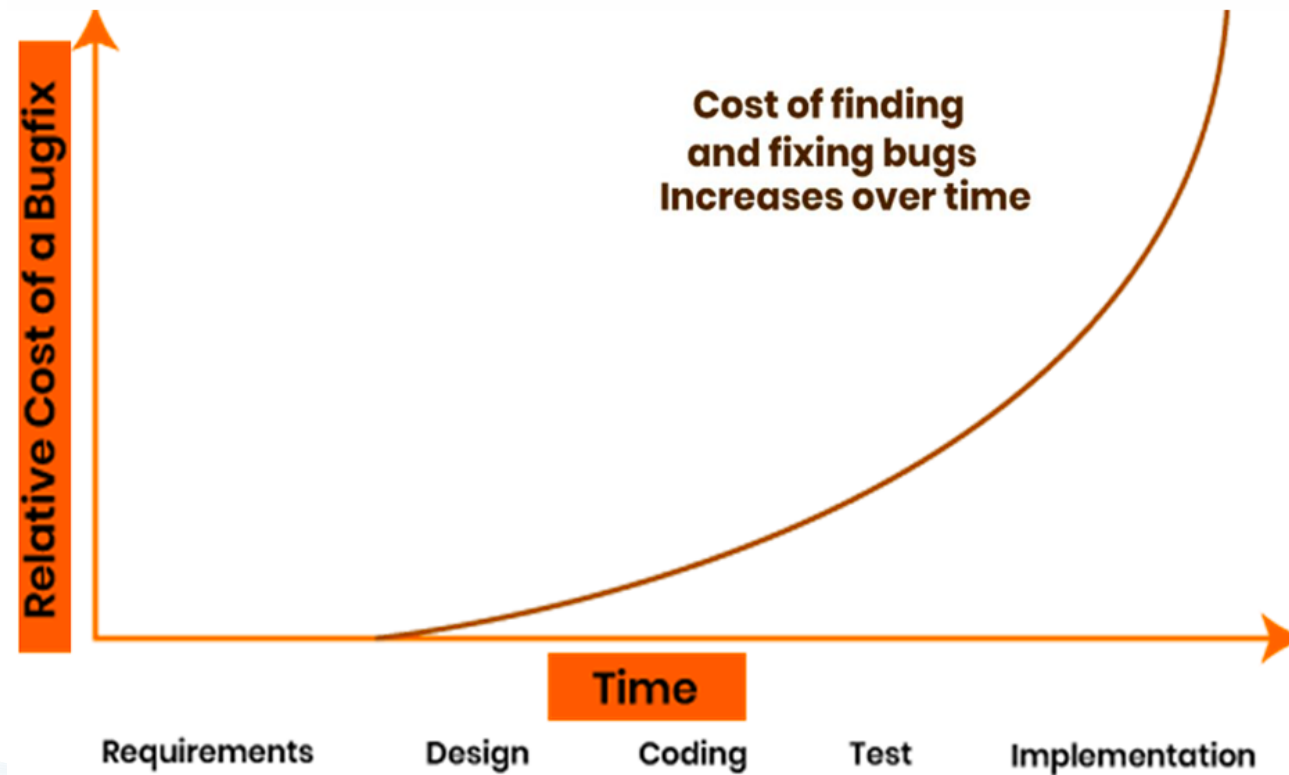
- ❑ Phần mềm là thành phần quan trọng trong các tổ chức, giúp quản lý và tự động hóa các chức năng hoạt động trong công ty. Các phần mềm có thể được phát triển bởi các cá nhân hoặc công ty, và nó có nguy cơ mắc lỗi và tạo ra các lỗ hổng.
- ❑ Các lỗ hổng có thể cho phép kẻ tấn công chạy mã, truy cập bộ nhớ của hệ thống, cài đặt phần mềm độc hại và đánh cắp, phá hủy hoặc sửa đổi những dữ liệu nhạy cảm. Đây là một trong những nguyên nhân hàng đầu gây ra các cuộc tấn công mạng nhắm vào tổ chức, doanh nghiệp và gây ra thiệt hại lên tới hàng ngàn tỉ USD trên toàn cầu.
- ❑ Do đó cần phải tạo ra sản phẩm phần mềm an toàn.

- ❑ Dòng mã (LOC), đo kích thước của chương trình máy tính
- ❑ Chương trình tốt hơn không có nghĩa là nhiều mã hơn

- ❑ Việc sửa một lỗi mất thời gian gấp nhiều lần so với việc viết một dòng mã.



Chi phí tìm và sửa lỗi thường tăng đáng kể khi lỗi được phát hiện muộn hơn trong vòng đời phát triển phần mềm.



❑ Trong phát triển phần mềm, các lỗi bảo mật phần mềm có thể là

- Lỗi bảo mật, lỗi, lỗ hổng, hoặc điểm yếu trong ứng dụng phần mềm.
- Đây có thể là lỗi thiết kế bảo mật phần mềm, lỗi mã hóa, lỗ hổng kiến trúc phần mềm hoặc lỗi triển khai.

❑ Các dạng lỗi bảo mật chương trình

- Cố ý, vô ý
- Bất cứ ở đâu: Hệ điều hành, phần cứng
- Bất cứ lúc nào: phát triển, bảo trì

❑ Tương tác không an toàn giữa các thành phần

- Ví dụ: đầu vào không hợp lệ, cross-site scripting, buffer overflow, lỗi chèn và xử lý lỗi không đúng cách

❑ Quản lý tài nguyên rủi ro

- Tràn bộ nhớ
- Giới hạn không đúng tên đường dẫn vào thư mục bị hạn chế
- Tải xuống mã không kiểm tra tính toàn vẹn

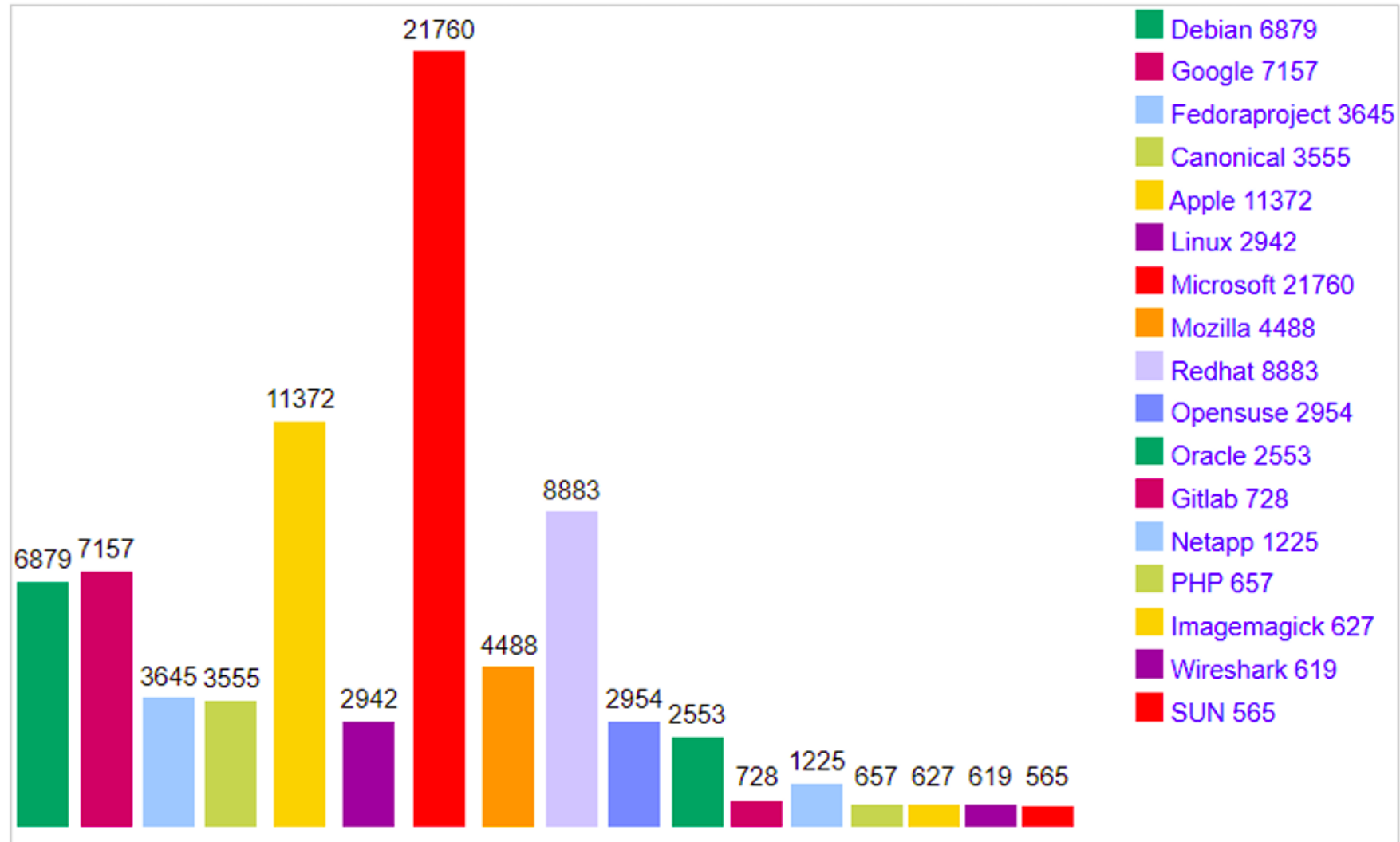
❑ Hàng phòng ngự bị rò rỉ

- Thiếu xác thực cho chức năng quan trọng
- Thiếu ủy quyền
- Sử dụng thông tin xác thực được mã hóa cứng
- Thiếu mã hóa dữ liệu nhạy cảm

- ❑ Lỗi trong ứng dụng hoặc cơ sở hạ tầng của nó:
 - Tức là không làm những gì nên làm
 - Ví dụ, cờ truy cập có thể được sửa đổi bởi đầu vào của người dùng
- ❑ Các tính năng không thích hợp trong cơ sở hạ tầng:
 - Tức là làm điều gì đó không nên làm
 - Ví dụ: một chức năng tìm kiếm có thể hiển thị thông tin người dùng khác
- ❑ Sự không phù hợp khi sử dụng các tính năng được cung cấp bởi cơ sở hạ tầng

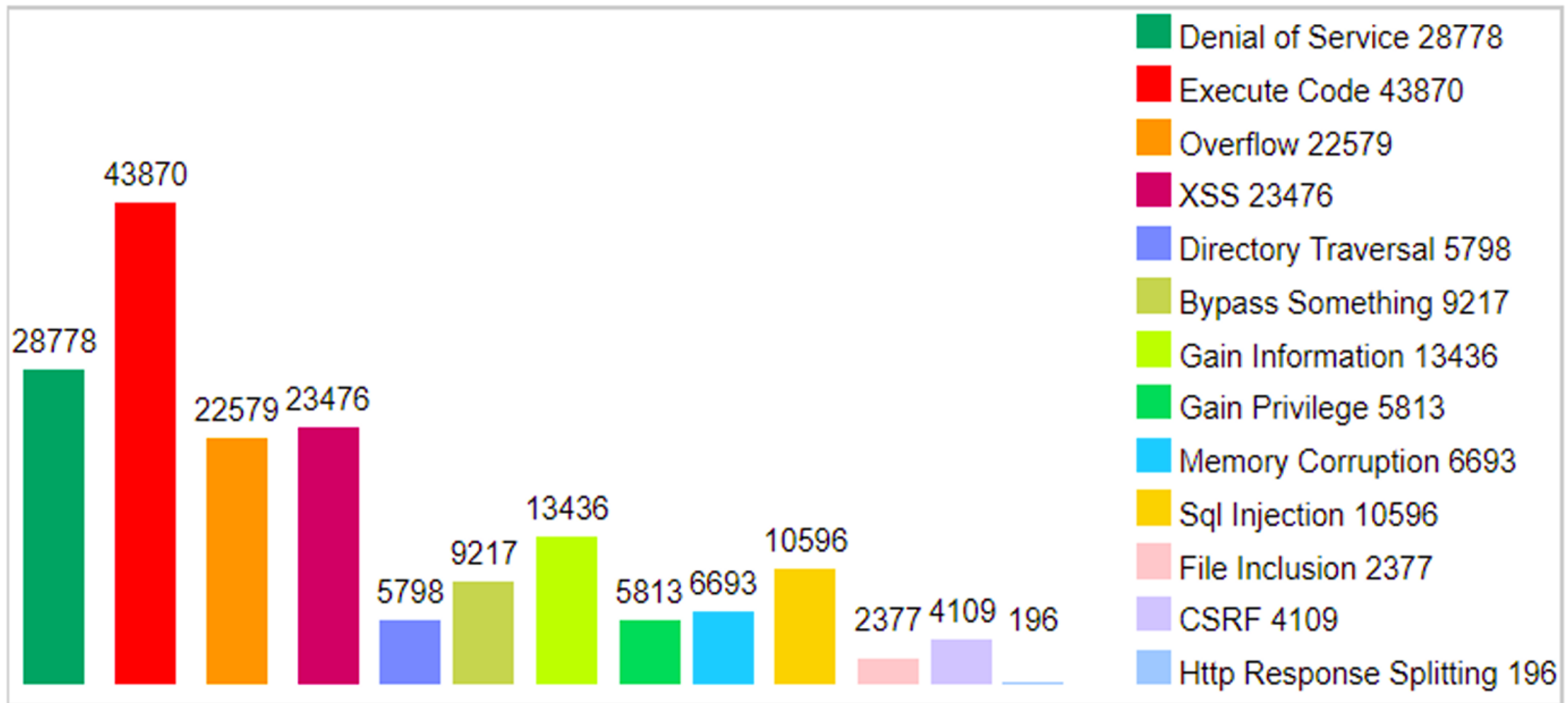
1. **Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')**
2. **Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')**
3. **Cross-Site Request Forgery (CSRF)**
4. **Missing Authorization**
5. **Out-of-bounds Write**
6. **Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')**
7. **Use After Free**
8. **Out-of-bounds Read**
9. **Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')**
10. **Improper Control of Generation of Code ('Code Injection')**

Total Number Of Vulnerabilities Of Top 50 Products By Vendor



Vulnerabilities By Type

<https://www.cvedetails.com>

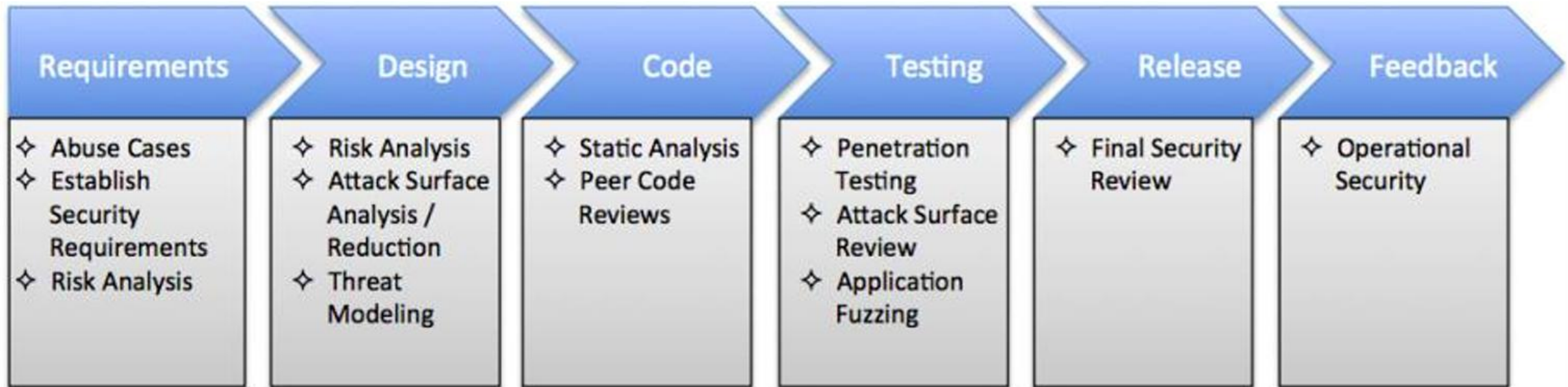


Cục ATTT thuộc Bộ Thông tin và Truyền thông (TTTT) thông báo các lỗ hổng (2023).

- ❑ CVE-2023-21674 trong “Windows Advanced Local Procedure Call” cho phép đối tượng tấn công thực hiện nâng cao đặc quyền.
- ❑ CVE-2023-21743, CVE-2023-21744, CVE-2023-21742 trong “Microsoft SharePoint Server”, cho phép đối tượng tấn công thực hiện tấn công vượt qua cơ chế bảo mật và thực thi mã từ xa.
- ❑ CVE-2023-21763, CVE-2023-21764, CVE-2023-21762, CVE-2023-21745 tồn tại ở phần mềm “Microsoft Exchange Server”. Trong đó, 2 lỗ hổng CVE-2023-21763, CVE-2023-21764 cho phép đối tượng tấn công thực hiện nâng cao đặc quyền. 2 lỗ hổng còn lại cho phép đối tượng tấn công thực hiện tấn công giả mạo.

- ☐ Một hình thức thiết kế phòng thủ để đảm bảo chức năng liên tục của phần mềm bất chấp việc sử dụng không lường trước được
- ☐ Yêu cầu chú ý đến tất cả các khía cạnh của việc thực hiện chương trình, môi trường, dữ liệu được xử lý.

Bảo mật nên là vấn đề tiên quyết ngay trong giai đoạn đầu xây dựng, phát triển phần mềm

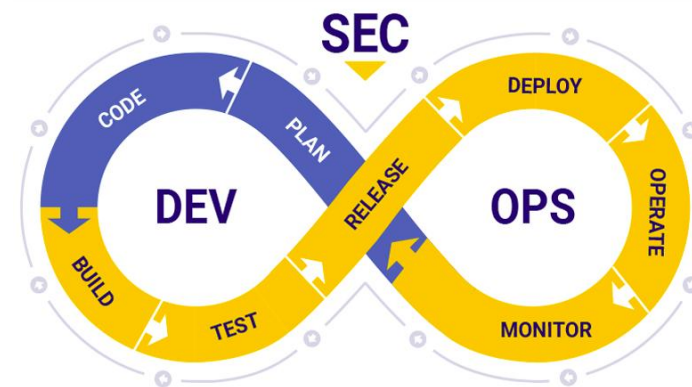


Integrating Security into the Software Development Life Cycle

© Capstone Security, Inc.

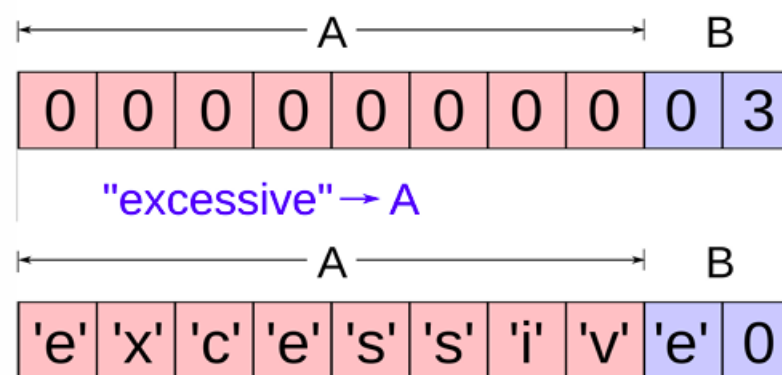
DevSecOps đặt vấn đề bảo mật lên hàng đầu trong toàn bộ quá trình phát triển, gồm một số vấn đề sau:

- Khung bảo mật (Security Framework)
- Đào tạo về lập trình an toàn
- Cơ chế cổng bảo mật: giúp xác định chính xác những lỗi hổng nghiêm trọng cần khắc phục trước khi phát hành phần mềm
- Thực hiện chiến lược bảo mật nhiều lớp
- Quản lý tốt việc sử dụng các thư viện bên thứ ba



Buffer Overflow, còn được gọi là buffer overrun hoặc buffer overwrite

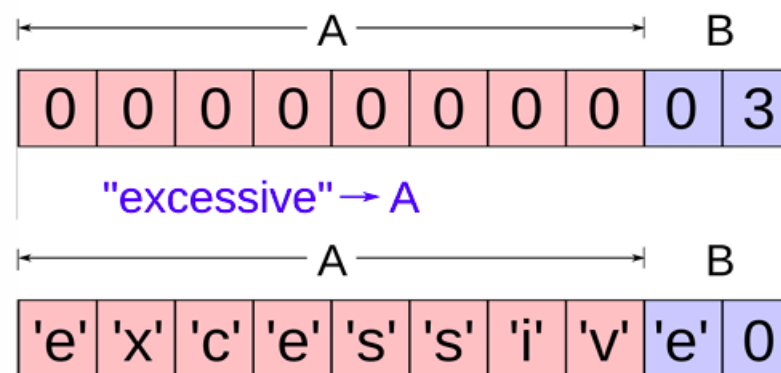
- Lỗi tràn bộ nhớ đệm là một điều kiện bất thường khi một tiến trình lưu dữ liệu vượt ra ngoài biên của một bộ nhớ đệm có chiều dài cố định. Kết quả là dữ liệu đó sẽ đè lên các vị trí bộ nhớ liền kề.
- Dữ liệu bị ghi đè có thể bao gồm các bộ nhớ đệm khác, các biến và dữ liệu điều khiển luồng chạy của chương trình.



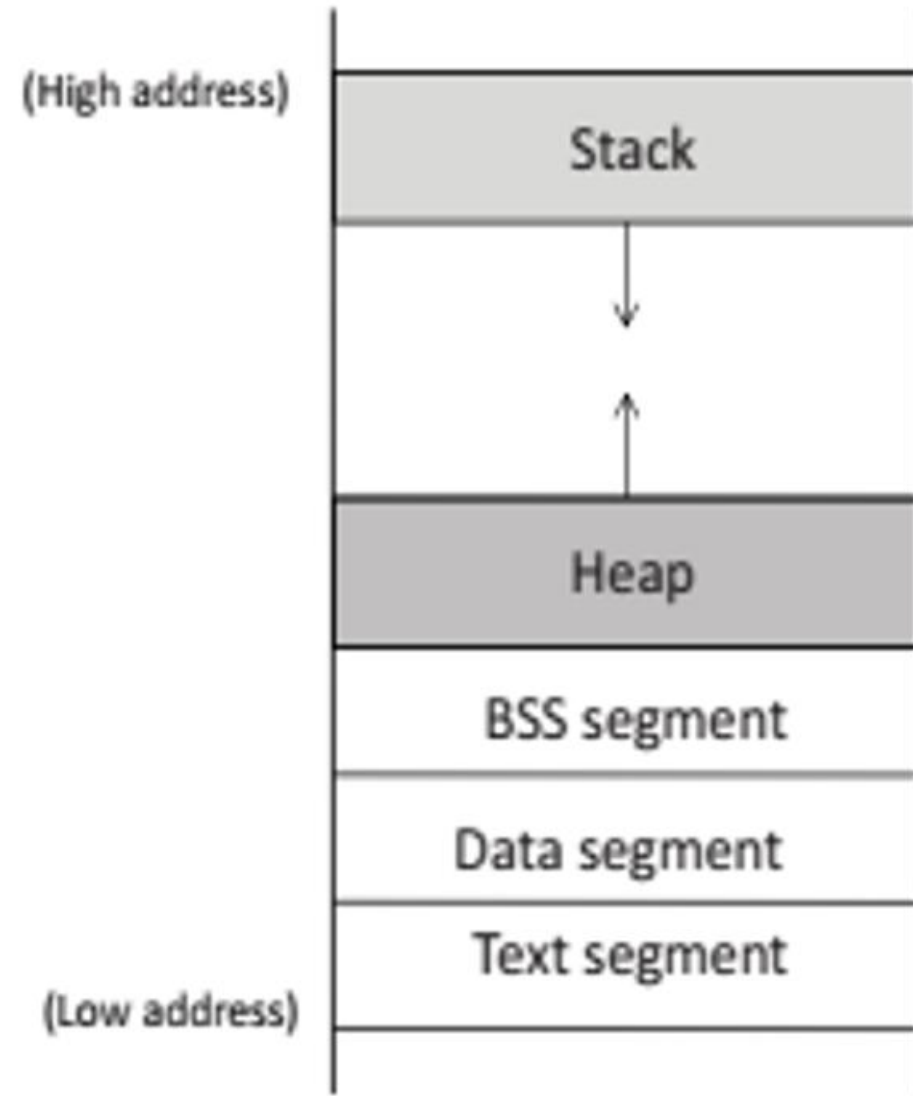
Có 2 loại buffer overflow

- Stack overflow: Tràn bộ đệm trên Stack
- Heap overflow: Tràn bộ đệm trên Heap.

Lỗi tràn bộ đệm gây ra nhiều lỗ hổng bảo mật đối với phần mềm và tạo cơ sở cho nhiều thủ thuật khai thác bảo mật



- ❑ **Stack:** lưu trữ các biến cục bộ trong func, lưu trữ dữ liệu liên quan đến các lệnh gọi hàm: địa chỉ trả về, đối số, (LIFO)
- ❑ **Heap:** cung cấp không gian cho việc phân bổ bộ nhớ động. Khu vực này được quản lý bởi malloc, calloc ...
- ❑ **BSS segment:** lưu trữ các biến tĩnh/toàn cục chưa được khởi tạo (mặc định bằng 0)
- ❑ **Data segment:** lưu trữ các biến tĩnh/toàn cục được lập trình viên khởi tạo
- ❑ **Text:** lưu trữ mã thực thi của chương trình (chỉ đọc)



☐ Stack Pointer:

- esp - Extended Stack Pointer

☐ Stack Pointer Register

☐ Stack Frame.

☐ Frame Pointer

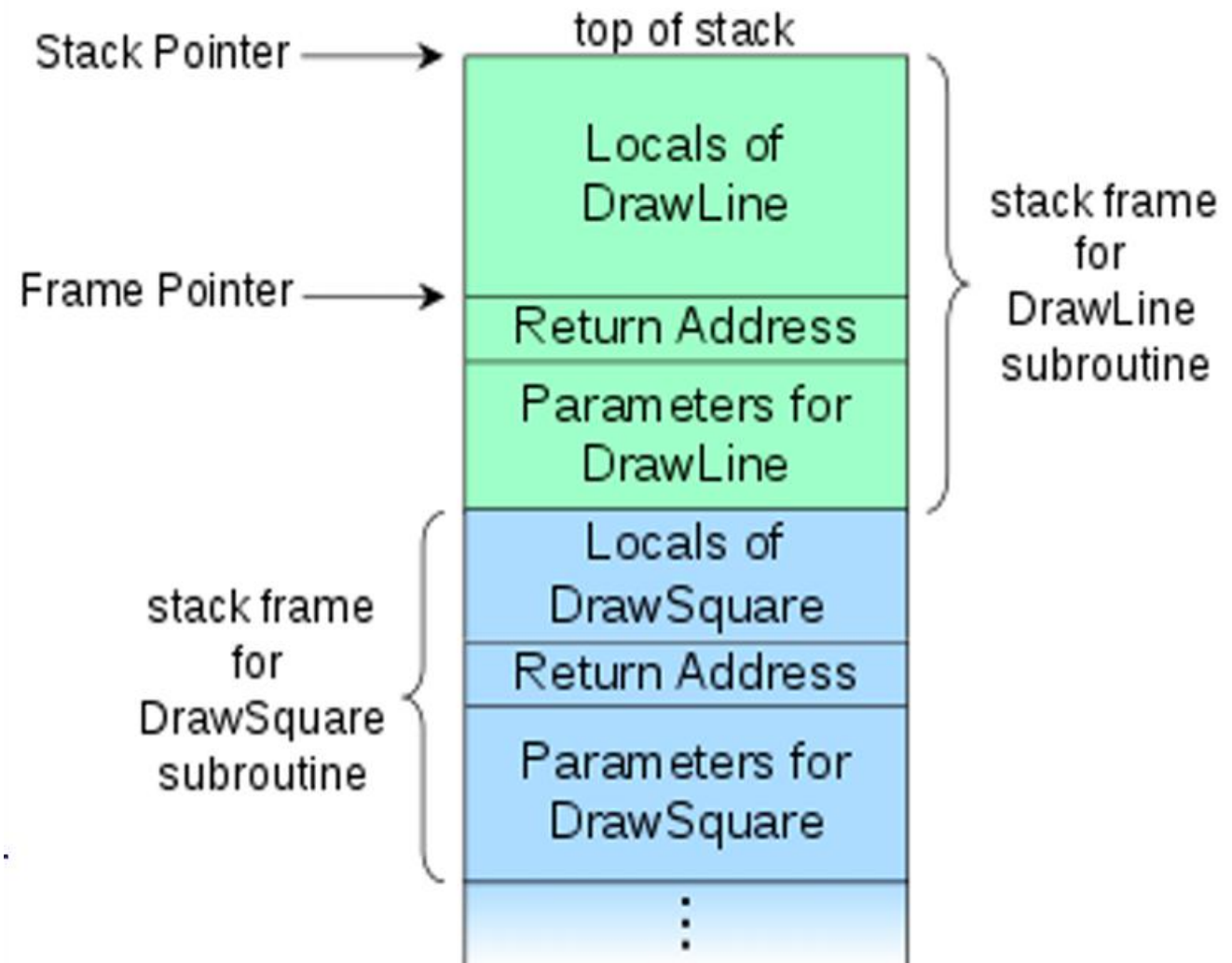
- ebp - Extended Base Pointer

☐ Frame Pointer Register.

☐ Return address

☐ The instruction pointer

- eip - Extended Instruction Pointer



- ❑ Stack Pointer (esp) tự động di chuyển khi nội dung được push or pop ra khỏi ngăn xếp.
- ❑ Stack Pointer Register: Lưu trữ địa chỉ bộ nhớ mà con trỏ stack (đỉnh hiện tại của stack: trỏ tới cuối bộ nhớ thấp) đang trỏ đến.
- ❑ Stack Frame: Bản ghi kích hoạt cho 1 thủ tục gồm (theo thứ tự hướng tới phía cuối bộ nhớ thấp): các tham số, địa chỉ trả về, con trỏ khung cũ, các biến cục bộ. – nó được tạo ra khi có lời gọi hàm => để lưu các biến trong hàm đó.

- ❑ **Frame Pointer:** thường chỉ đến 1 địa chỉ cố định, sau địa chỉ (đối mặt với bộ nhớ thấp kết thúc) nơi con trỏ khung cũ được lưu trữ
- ❑ **Frame Pointer Register:** lưu trữ địa chỉ bộ nhớ mà con trỏ khung (tham chiếu cho một SF đối với vị trí bộ nhớ khác nhau có thể được truy cập bằng cách sử dụng địa chỉ tương đối) được trỏ đến.
- ❑ **Địa chỉ trả về:** Địa chỉ bộ nhớ mà điều khiển thực hiện sẽ trở lại khi quá trình thực hiện của một ngăn xếp được hoàn thành

Ví dụ

```
#include <stdio.h>

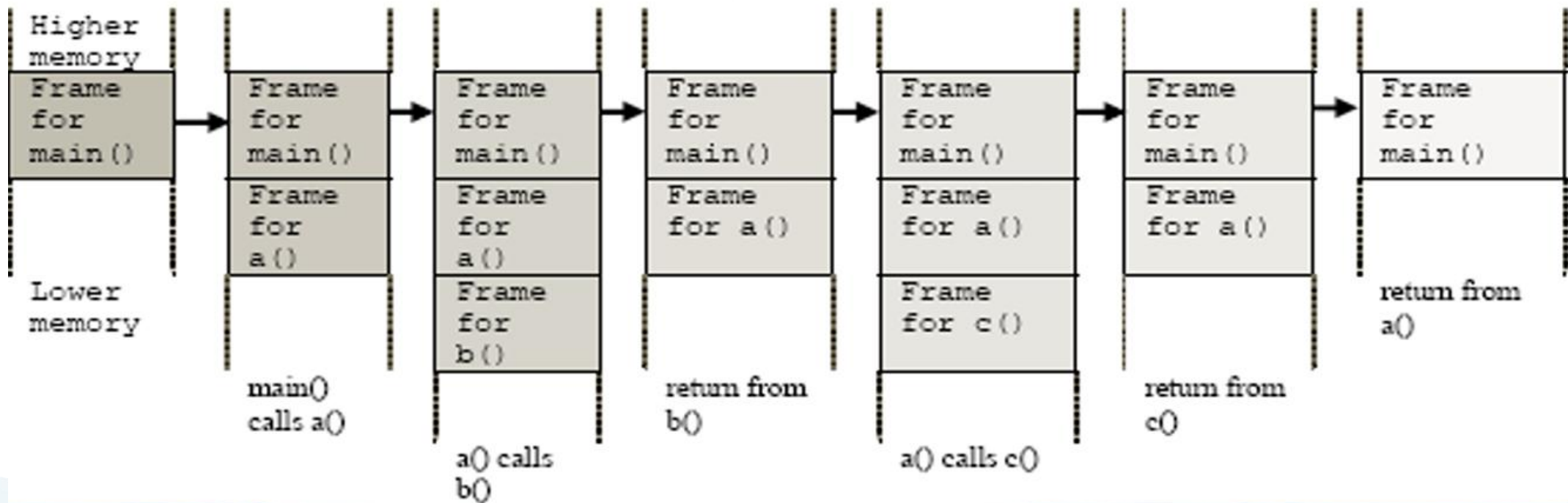
int a();
int b();
int c();
```

```
int a()
{
    b();
    c();
    return 0;
}
```

```
int b()
{ return 0;
}
```

```
int c()
{ return 0;
}
```

```
int main()
{
    a();
    return 0;
}
```



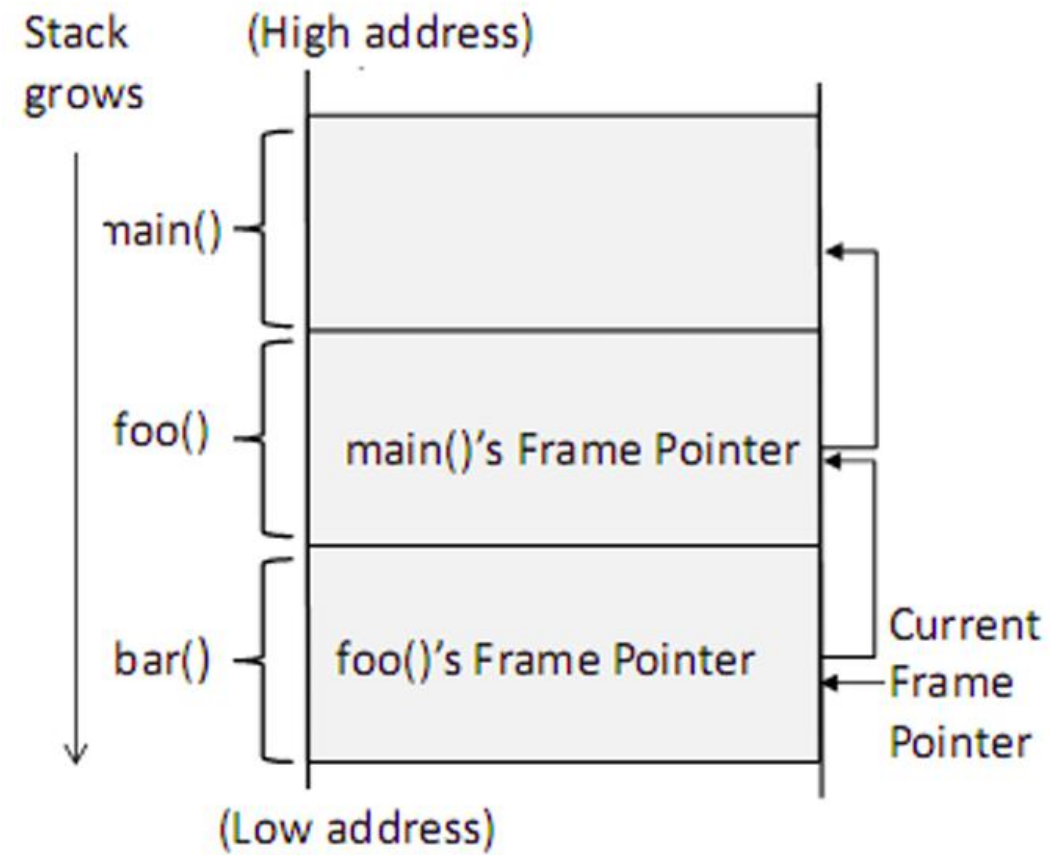
Khi một hàm được gọi, một khối không gian bộ nhớ sẽ được phân bổ ở đầu ngăn xếp và nó được gọi là khung ngăn xếp

- ❑ **Đối số:** lưu trữ giá trị của các đối số được truyền vào hàm. ví dụ: (5 ,8) sẽ được đẩy vào ngăn xếp - phần đầu của stack frame
- ❑ **Địa chỉ trả về:** khi hàm kết thúc và gặp lệnh trả về, nó cần biết nơi để quay về.
 - Trước khi nhảy tới đầu vào của hàm, máy tính sẽ đẩy địa chỉ của lệnh tiếp theo (lệnh được đặt ngay sau lệnh gọi hàm) vào đầu ngăn xếp, tức là vùng "địa chỉ trả về" trong khung ngăn xếp.

- ❑ Con trỏ khung trước: Mục tiếp theo được chương trình đẩy vào khung ngăn xếp là con trỏ khung cho khung trước đó.
- ❑ Biến cục bộ: lưu trữ các biến cục bộ của hàm (tùy thuộc vào trình biên dịch, ví dụ ngẫu nhiên hóa thứ tự của các biến cục bộ)
- ❑ Con trỏ khung: giúp truy cập các đối số và các biến cục bộ trong func.

Arguments
Return Address
Previous Frame Pointer
Local Variables

- ❑ Chỉ có một thanh ghi con trỏ khung và nó luôn trỏ đến khung ngăn xếp của hàm hiện tại
 - ❑ Vấn đề: khi chúng ta trả về từ `bar()`, chúng ta sẽ không thể biết khung ngăn xếp của hàm `foo()` ở đâu
 - ❑ Giải pháp: trước khi nhập hàm được gọi, giá trị con trỏ khung của người gọi được lưu trữ trong trường "con trỏ khung trước đó" trên ngăn xếp
- => giá trị trong trường này sẽ được sử dụng để thiết lập thanh ghi con trỏ khung



Tràn bộ đệm: (lỗi lập trình)

- ☐ Cố gắng lưu trữ dữ liệu vượt quá giới hạn của bộ đệm có kích thước cố định. ghi đè lên các vị trí bộ nhớ liền kề: có thể chứa các biến hoặc tham số chương trình khác hoặc dữ liệu luồng điều khiển chương trình như địa chỉ trả về và con trỏ đến các khung ngăn xếp trước đó.
- ☐ Bộ đệm có thể nằm: trên stack, trong heap hoặc trong phần dữ liệu của quy trình.
- ☐ Hậu quả của lỗi này bao gồm: hỏng dữ liệu mà chương trình sử dụng, chuyển giao quyền điều khiển bất ngờ trong chương trình, có thể vi phạm quyền truy cập bộ nhớ và rất có thể là chương trình sẽ kết thúc.

- ❑ Kể từ năm 1988, tràn stack đã dẫn đến những thỏa hiệp nghiêm trọng nhất về an ninh.
- ❑ Vấn đề tràn bộ đệm có lịch sử lâu dài, gây ra các tấn công nghiêm trọng
- ❑ Hiện nay, hầu hết các OS đều đã phát triển các biện pháp đối phó với lỗi tràn bộ đệm, và do đó hiệu quả của các kỹ thuật tràn stack truyền thống sẽ giảm đi.
- ❑ Để đơn giản hóa các thực nghiệm về lỗi tràn stack, trước tiên chúng ta cần tắt các biện pháp đối phó này.

□ Hai loại tràn Stack:

- Stack smash, ghi đè lên con trỏ lệnh đã lưu (eip- Instruction Pointer) không kiểm tra độ dài của dữ liệu được cung cấp và chỉ đơn giản là đặt nó vào một bộ đệm có kích thước cố định
- Stack off-by-one, ghi đè lên con trỏ khung đã lưu (ebp- Frame Pointer) lỗi lập trình liên quan đến tính toán độ dài của các chuỗi trong 1 chương trình

Đặt dữ liệu vào một bộ đệm có kích thước cố định

A Stack Frame of main(): ex1.c

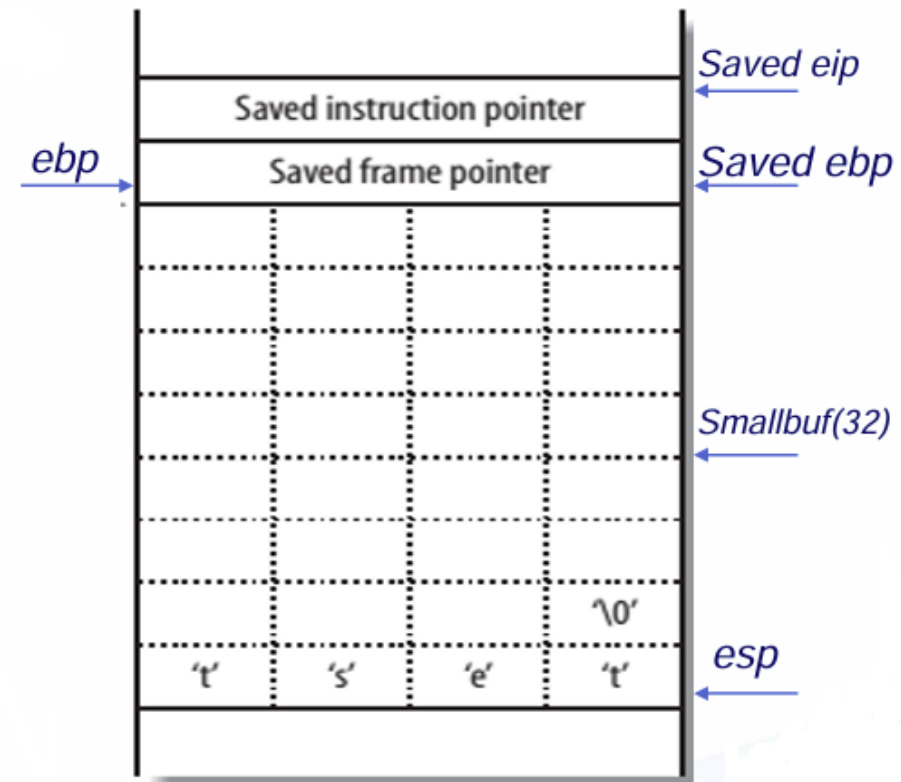
Code:

```
int main(int argc, char *argv[])
{
    char smallbuf[32];
    strcpy(smallbuf, argv[1]);
    printf("%s\n", smallbuf);
    return 0;
}
```

Compile: gcc -o ex1.out ex1.c

Run:

```
./ex1.out test
./ex1.out $(python -c "print('a'*5)")
```

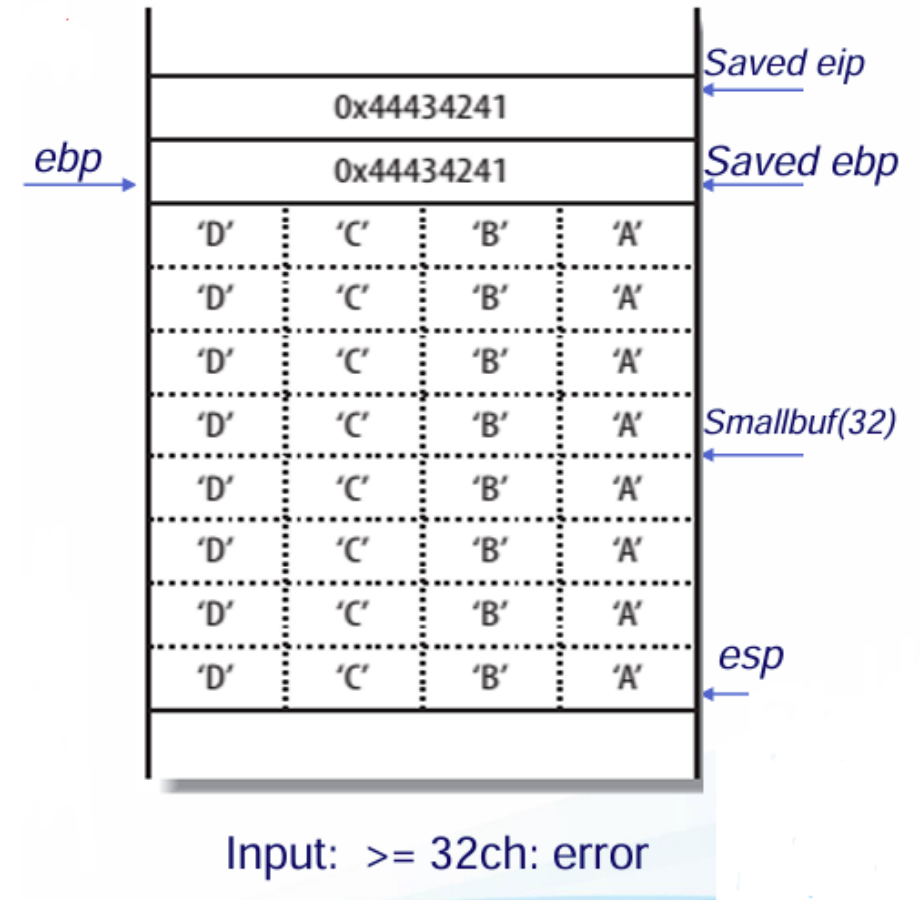


Input: test < 32ch: ok

Lỗi phân đoạn xảy ra khi hàm main() trả về, giá trị của buff tràn và ghi đè lên ebp và đè lên eip .

Thực hiện:

- ❑ Đẩy giá trị 0x44434241 (“DCBA” - hệ hexadecimal) ra khỏi stack
- ❑ Cố gắng tìm nạp, giải mã và thực hiện lệnh tại địa chỉ đó – đã bị ghi đè (bởi 0x44434241 - không chứa lệnh hợp lệ) => chương trình crashing, hoặc thực thi mã lệnh bất kỳ



- ❑ Thi hành mã tùy ý: kẻ tấn công có thể thực thi mã tùy ý với quyền của tiến trình bị ảnh hưởng. Điều này có thể dẫn đến sự xâm phạm hoàn toàn hệ thống, bao gồm cả việc đánh cắp thông tin nhạy cảm.
- ❑ DoS: Lỗi tràn bộ đệm có thể khiến chương trình bị treo hoặc bị treo, dẫn đến tấn công từ chối dịch vụ. Nó cũng có thể gây ra phản ứng dây chuyền lan sang các hệ thống và dịch vụ khác, có khả năng dẫn đến tình trạng ngừng hoạt động trên diện rộng.

- ☐ Rò rỉ thông tin: Kẻ tấn công có thể truy cập thông tin nhạy cảm, chẳng hạn như mật khẩu, khóa mã hóa hoặc dữ liệu bí mật, được lưu trữ trong bộ nhớ của quy trình bị ảnh hưởng.
- ☐ Leo thang đặc quyền: Kẻ tấn công có thể giành được các đặc quyền nâng cao trên hệ thống bị ảnh hưởng, có khả năng vượt qua các biện pháp kiểm soát bảo mật và truy cập vào các hệ thống và dữ liệu nhạy cảm

- ☐ Khám phá: Xác định lỗi hổng có thể đc thực hiện thông qua đánh giá mã thủ công, quét lỗi hổng tự động hoặc các phương tiện khác.
- ☐ Tạo payload: kẻ tấn công phải tạo một payload sẽ ghi đè lên bộ đệm và chuyển hướng luồng thực thi của chương trình sang mã của kẻ tấn công. Kẻ tấn công phải xây dựng cẩn thận payload này để vượt qua mọi tính năng bảo mật như ASLR.

- ❑ **Chèn:** Sau đó, payload được đưa vào bộ đệm, thường thông qua vector tấn công dựa trên mạng, chẳng hạn như gói mạng hoặc yêu cầu web.
- ❑ **Kích hoạt:** Kẻ tấn công sau đó phải kích hoạt tình trạng tràn bộ đệm, khiến chương trình ghi payload vào bộ đệm và ghi đè lên các vị trí bộ nhớ lân cận.
- ❑ **Thực thi:** Khi payload đã được đưa vào và lỗi tràn bộ đệm được kích hoạt, mã của kẻ tấn công sẽ được thực thi, cho phép chúng kiểm soát chương trình và thực hiện các hành động mong muốn của chúng

Một hàm lồng nhau để thực hiện sao chép chuỗi vào bộ đệm.

Xét ví dụ trước. Nếu chuỗi dài hơn 32 ký tự, nó không được xử lý.

Input:

> 32 : -> Input string too long!

<32 : -> printf

=32 : Segmentation fault (core dumped)

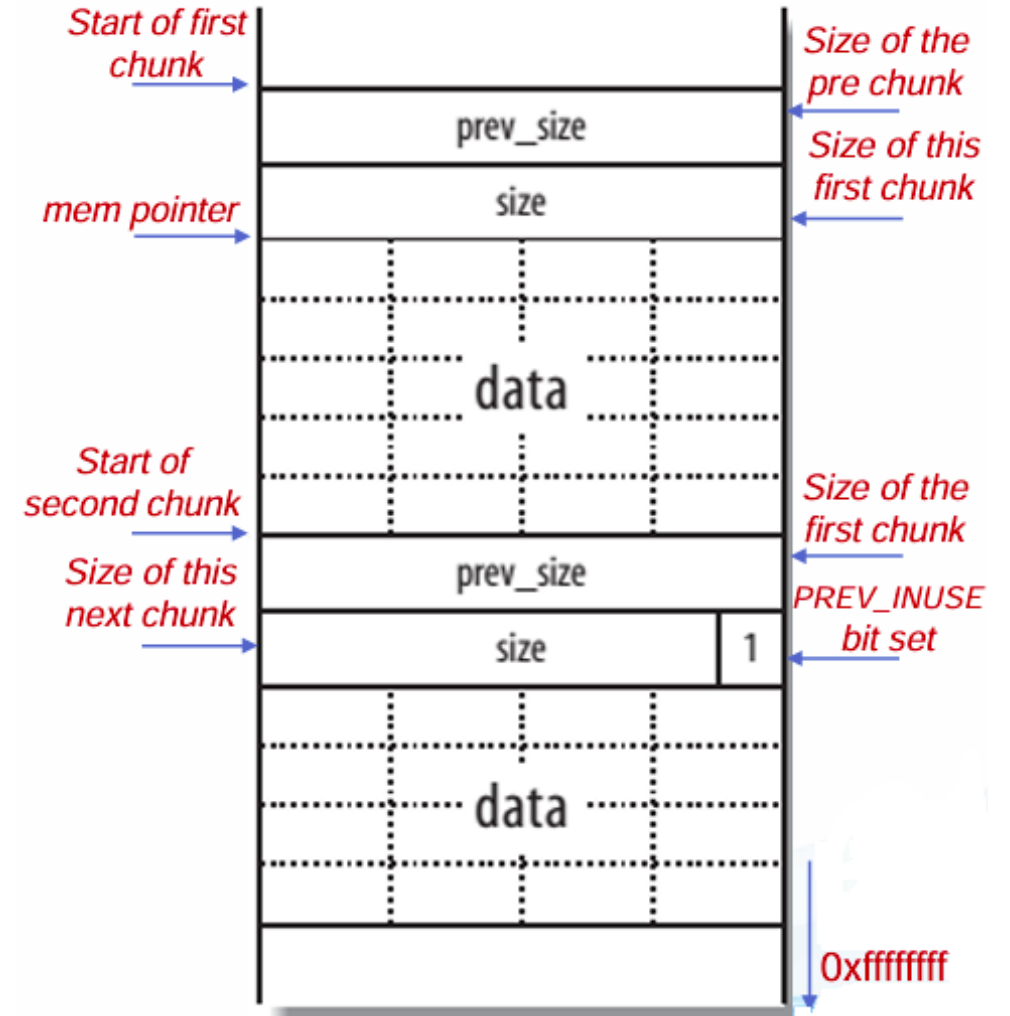
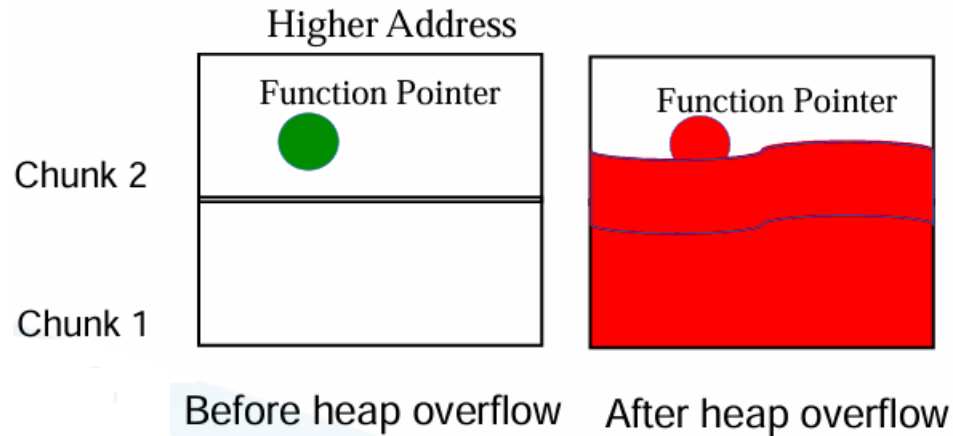
```
Code: ex2.c
int main(int argc, char *argv[])
{
    if(strlen(argv[1]) > 32)
        {printf("Input string too
long!\n");
        exit (1);
        }
    vulfunc(argv[1]);
    return 0;
}
int vulfunc(char *arg)
{
    char smallbuf[32];
    strcpy(smallbuf, arg);
    printf("%s\n", smallbuf);
    return 0;
}
```

```
# ./ex2 test
test
# ./ex2 ABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCD
Input string too long!
# ./ex2 ABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCD
ABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCD
# ./ex2 ABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCD
ABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCDABCD
Segmentation fault (core dumped)
```

- ❑ Bộ nhớ Heap: Một app không biết chọn size cho buff khi nó đang chạy=> dùng heap để tự động phân bổ buffer với các kích cỡ khác nhau.
- ❑ Heap overflow: Các bộ đệm Heap này dễ bị tràn nếu dữ liệu do người dùng cung cấp không được kiểm tra, dẫn đến tấn công ghi đè lên các giá trị khác trên heap có thể dẫn đến việc xâm phạm dữ liệu nhạy cảm (ghi đè tên tệp và các biến khác) luồng chương trình logic (thông qua cấu trúc điều khiển heap và sửa đổi con trỏ hàm)
- ❑ Các hàm thao tác heap điển hình: `malloc()/free()`

Heap được chia thành các chunk

- ❑ Mỗi chunk có các thành phần như hình
- ❑ Dữ liệu nằm theo từng chunk trên Heap
- ❑ Ghi đè con trỏ hàm trong bộ đệm liên kề



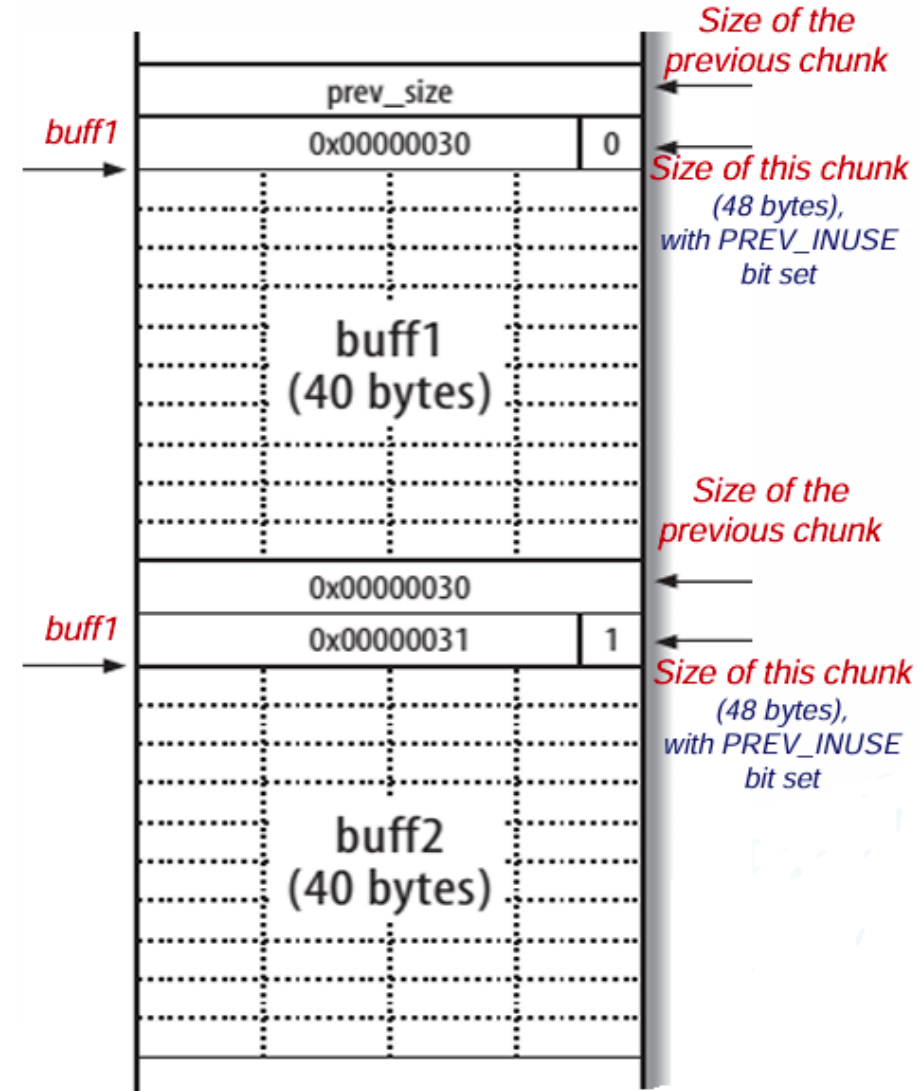
Ví dụ.

```
intmain(void)
{
char *buff1, *buff2;
buff1 = malloc(40);
buff2 = malloc(40);
gets(buff1);
free(buff1);
exit(0);
}
```

There is no checking imposed on the data fed into buff1 by gets(),

=> a heap overflow can occur.

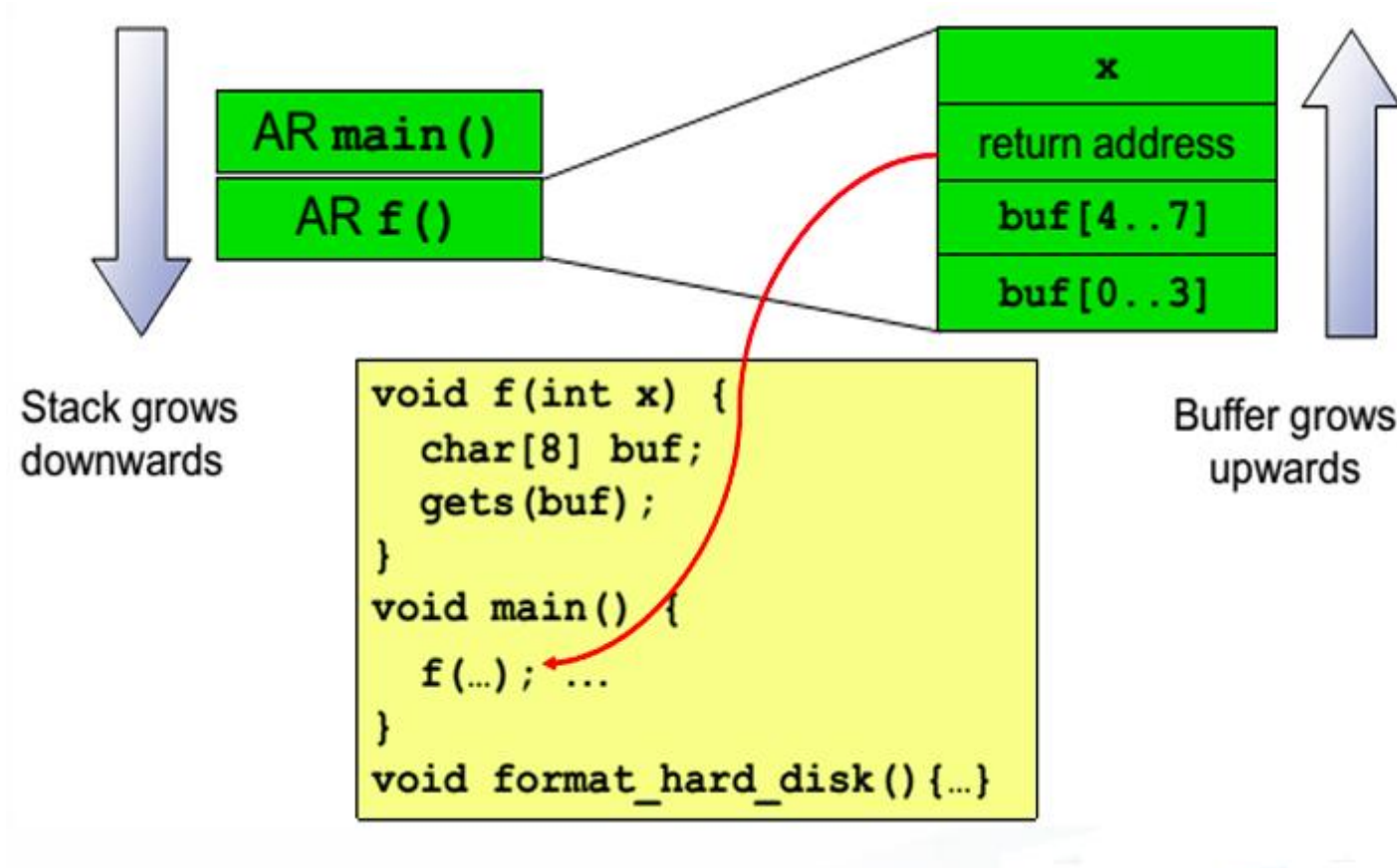
Warning:the 'gets' function is dangerous and should not be used”.



- ❑ Tấn công chèn mã: kẻ tấn công chèn mã shellcode của riêng mình vào bộ đệm và làm hỏng địa chỉ trả về trở đến mã này
 - Ví dụ: `exec (/bin/sh)`
 - Đây là cuộc tấn công tràn bộ đệm “cổ điển” [Smashing the stack để giải trí và kiếm lợi nhuận, Aleph One, 1996]
- ❑ Tấn công tái sử dụng mã: kẻ tấn công làm hỏng địa chỉ trả về để trở đến mã hiện có
 - Ví dụ: `format_hard_disk`
 - Vì sẽ được chèn trực tiếp vào giữa bộ nhớ chương trình để thực thi tiếp nên đoạn mã phải được viết ở dạng hợp ngữ

Ex, format_hard_disk

Stack bao gồm các Bản ghi kích hoạt- Activation Records



- ☐ Sử dụng ngôn ngữ an toàn
- ☐ No execute stack
- ☐ Ngẫu nhiên hóa không gian địa chỉ
- ☐ Canaries
- ☐ Tránh các thư viện xấu đã biết

❑ Sử dụng ngôn ngữ an toàn

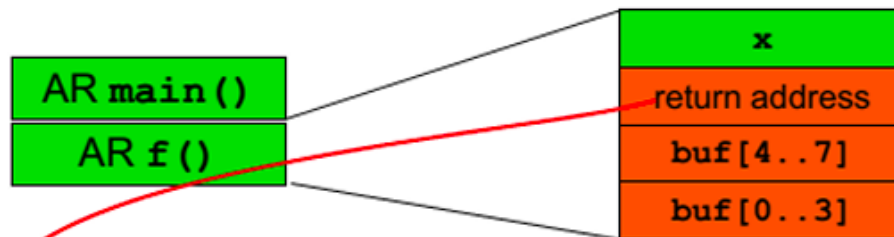
- Tràn bộ đệm trở nên không thể thực hiện được do kiểm tra hệ thống runtime, khả năng suy giảm hiệu suất
- Java, C++, Python

❑ Khi sử dụng ngôn ngữ không an toàn:

- Kiểm tra đầu vào (ALL input is EVIL)
- Sử dụng các chức năng an toàn hơn để kiểm tra ranh giới
- Sử dụng các công cụ tự động để phân tích mã cho các chức năng tiềm ẩn không an toàn.
- Có thể đánh dấu các chức năng hoặc cấu trúc tiềm ẩn không an toàn
- Có thể giúp giảm thiểu sự mất an ninh, nhưng thực sự là khó để loại bỏ tất cả các tràn bộ đệm

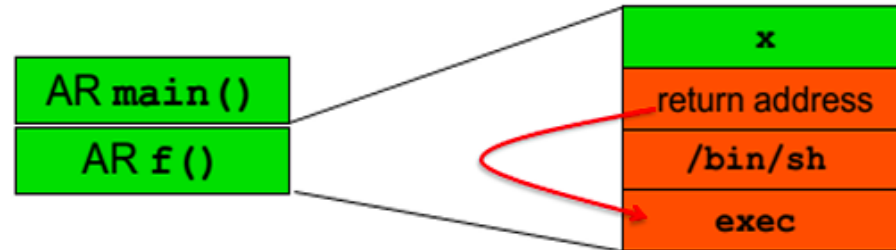
Ví dụ:

What if **gets()** reads more than 8 bytes ?
Attacker can jump to any point in the code!



```
void f(int x) {  
    char[8] buf;  
    gets(buf);  
}  
void main() {  
    f(...); ...  
}  
void format_hard_disk() {...}
```

What if **gets()** reads more than 8 bytes ?
Attacker can even jump to his own code in buffer! (shell code)



```
void f(int x) {  
    char[8] buf;  
    gets(buf);  
}  
void main() {  
    f(...); ...  
}  
void format_hard_disk() {...}
```

Never use
gets()

Stack Canaries (hệ thống cảnh báo)

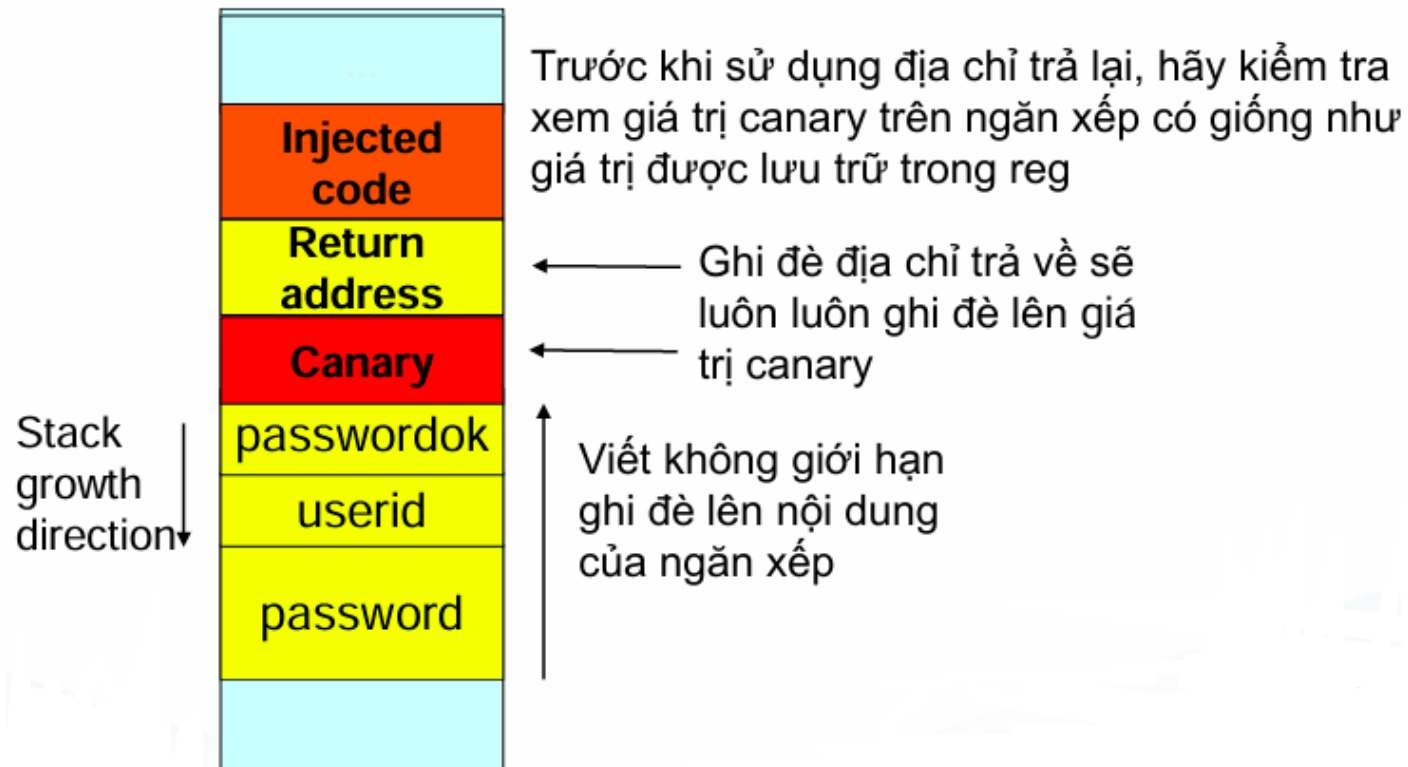
❑ 1 giá trị canary random được viết ngay trước khi 1 địa chỉ trả về được lưu trữ trong 1 khung stack (giá trị được chọn random khi bắt đầu chương trình)

=> Khi cố ghi đè Re Addr bằng lỗi overflow sẽ dẫn đến việc canary đang bị ghi đè và lỗi tràn sẽ được phát hiện. => phát hiện lỗi tràn bộ nhớ đệm trước khi 1 mã độc được thực thi.

=> Làm khó cho attacker khi sử dụng lỗi tràn bộ nhớ đệm vì nó buộc attacker phải đoạt quyền kiểm soát con trỏ bằng những phương pháp không cổ điển như phá hỏng những biến quan trọng khác trên bộ nhớ đệm.

Stack Canaries (hệ thống cảnh báo)

- ❑ Canary để phát hiện giả mạo
- ❑ Không thực thi mã trên ngăn xếp



- ❑ Ngẫu nhiên không gian địa chỉ - Address space randomization: Ubuntu và các bản phân phối Linux khác đã triển khai một số cơ chế bảo mật để khiến cuộc tấn công tràn bộ đệm trở nên khó khăn.
- ❑ Các cơ chế bảo mật:
 - Ngẫu nhiên hóa sơ đồ không gian địa chỉ (ASLR- Address Space Layout Randomization)
 - Sơ đồ bảo vệ StackGuard
 - Sử dụng ngăn xếp không thể thực thi
- ❑ Chú ý: Để đơn giản hóa việc thực hành khai thác các lỗi tràn bộ đệm, trước tiên ta cần vô hiệu hóa các cơ chế bảo mật trên

Ngẫu nhiên hóa sơ đồ không gian địa chỉ - ASLR

- ☐ Ubuntu và một số Linux khác sử dụng ngẫu nhiên không gian địa chỉ để chọn ngẫu nhiên địa chỉ bắt đầu của heap và stack.
- ☐ Điều này làm cho việc đoán địa chỉ chính xác trở nên khó khăn (đ đoán địa chỉ là một trong những bước quan trọng của các cuộc tấn công tràn bộ đệm).
- ☐ Tính năng ngẫu nhiên hóa địa chỉ bị tắt, cho biết địa chỉ bắt đầu của ngăn xếp luôn giống nhau.
- ☐ Vô hiệu hóa các tính năng này bằng các lệnh sau:

```
sysctl -w kernel.randomize_va_space=0
```


Sơ đồ bảo vệ StackGuard

- ❑ Trình biên dịch GCC triển khai một cơ chế bảo mật được gọi là "Stack Guard" để ngăn chặn tràn bộ đệm.
- ❑ Khi có sự bảo vệ này, tràn bộ đệm sẽ không hoạt.
- ❑ Vô hiệu hóa các tính năng này bằng các lệnh sau:

-fno-stack-protector

Ex: *gcc -fno-stack-protector example.c*

Non-Executable Stack

Ubuntu từng cho phép các ngăn xếp thực thi được, nhưng điều này hiện đã thay đổi:

- ☐ Hình ảnh nhị phân của các chương trình (và thư viện dùng chung) phải khai báo xem chúng có yêu cầu ngăn xếp thực thi hay không, tức là chúng cần đánh dấu một trường trong tiêu đề chương trình.
- ☐ Kernel hoặc trình liên kết động sử dụng dấu hiệu này để quyết định xem có làm cho ngăn xếp của chương trình đang chạy này có thể thực thi được hay không.
- ☐ Việc đánh dấu này được thực hiện tự động bởi các phiên bản gcc gần đây và theo mặc định, ngăn xếp được đặt thành không thể thực thi được.

Non-Executable Stack

Để thay đổi tính năng này bằng các lệnh sau:

- Đối với ngăn xếp thực thi:

Ex: *\$ gcc -z execstack -o test test.c*

- Đối với ngăn xếp không thực thi:

Ex: *\$ gcc -z noexecstack -o test test.c*

- ☐ Vấn đề bảo mật phần mềm.
- ☐ Nguồn của lỗ hổng phần mềm
- ☐ Bố trí bộ nhớ xử lý
- ☐ Lỗ hổng phần mềm - Tràn bộ đệm: Stack overflow; Heap overflow
- ☐ Tấn công: chèn mã và tái sử dụng mã
- ☐ Các biến thể của tràn bộ đệm
- ☐ Phòng chống các cuộc tấn công tràn bộ đệm xếp

Biến cục bộ trong một hàm lưu ở đâu?

- a. Stack**
- b. Heap**
- c. BSS segment**
- d. Data segment**

Biến toàn cục bộ của chương trình đã được khởi tạo lưu ở đâu?

- a. Stack**
- b. Heap**
- c. BSS segment**
- d. Data segment**

Biến toàn cục bộ của chương trình chưa được khởi tạo lưu ở đâu?

- a. Stack**
- b. Heap**
- c. BSS segment**
- d. Data segment**

Biến con trỏ khi được cấp phát bộ nhớ được khởi lưu ở đâu?

- a. Stack**
- b. Heap**
- c. BSS segment**
- d. Data segmen**



AN TOÀN THÔNG TIN (INSE330380)

Kết thúc chương 2.1

Ths. ĐỖ MINH HẬU



Trung tâm Học liệu và Dạy học số
Đại học Công nghệ Kỹ thuật Tp. Hồ Chí Minh
01 Võ Văn Ngân, P. Thủ Đức, Tp. Hồ Chí Minh
daotaotuxa@hcmute.edu.vn